

Block Builder and Block Iterator

 This is a legacy version of the Mini-LSM course and we will not maintain it anymore. We now have a better version of this course and this chapter is now part of [Mini-LSM Week 1 Day 3: Blocks](#).

- [Task 1 - Block Builder](#)
- [Task 2 - Block Iterator](#)
- [Extra Tasks](#)

In this part, you will need to modify:

- `src/block/builder.rs`
- `src/block/iterator.rs`
- `src/block.rs`

You can use `cargo x copy-test day1` to copy our provided test cases to the starter code directory. After you have finished this part, use `cargo x scheck` to check the style and run all test cases. If you want to write your own test cases, write a new module `#[cfg(test)] mod user_tests { /* your test cases */ }` in `block.rs`. Remember to remove `#![allow(...)]` at the top of the modules you modified so that cargo clippy can actually check the styles.

Task 1 - Block Builder

Block is the minimum read unit in LSM. It is of 4KB size in general, similar to database pages. In each block, we will store a sequence of sorted key-value pairs.

You will need to modify `BlockBuilder` in `src/block/builder.rs` to build the encoded data and the offset array. The block contains two parts: data and offsets.

```
-----
| data | offsets | meta | | | | | |
| ---- | - - - - | ---- |
| entry | entry | entry | entry | offset | offset | offset | offset |
num_of_elements |
-----
```

When user adds a key-value pair to a block (which is an entry), we will need to serialize it into the following format:

	Entry #1			...

key_len (2B)	key (keylen)	value_len (2B)	value (varlen)	...

Key length and value length are both 2 bytes, which means their maximum lengths are 65535. (Internally stored as `u16`)

We assume that keys will never be empty, and values can be empty. An empty value means that the corresponding key has been deleted in the view of other parts of the system. For the `BlockBuilder` and `BlockIterator` , we just treat the empty value as-is.

At the end of each block, we will store the offsets of each entry and the total number of entries. For example, if the first entry is at 0th position of the block, and the second entry is at 12th position of the block.

	offset		offset	num_of_elements

	0		12	2

The footer of the block will be as above. Each of the number is stored as `u16` .

The block has a size limit, which is `target_size` . Unless the first key-value pair exceeds the target block size, you should ensure that the encoded block size is always less than or equal to `target_size` . (In the provided code, the `target_size` here is essentially the `block_size`)

The `BlockBuilder` will produce the data part and unencoded entry offsets when `build` is called. The information will be stored in the `Block` struct. As key-value entries are stored in raw format and offsets are stored in a separate vector, this reduces unnecessary memory allocations and processing overhead when decoding data — what you need to do is to simply copy the raw block data to the `data` vector and decode the entry offsets every 2 bytes, *instead of* creating something like `Vec<(Vec<u8>, Vec<u8>)>` to store all the key-value pairs in one block in memory. This compact memory layout is very efficient.

For the encoding and decoding part, you'll need to modify `Block` in `src/block.rs` . Specifically, you are required to implement `Block::encode` and `Block::decode` , which will encode to / decode from the data layout illustrated in the above figures.

Task 2 - Block Iterator

Given a `Block` object, we will need to extract the key-value pairs. To do this, we create an iterator over a block and find the information we want.

`BlockIterator` can be created with an `Arc<Block>`. If `create_and_seek_to_first` is called, it will be positioned at the first key in the block. If `create_and_seek_to_key` is called, the iterator will be positioned at the first key that is `>=` the provided key. For example, if `1`, `3`, `5` is in a block.

```
let mut iter = BlockIterator::create_and_seek_to_key(block, b"2");
assert_eq!(iter.key(), b"3");
```

The above `seek 2` will make the iterator to be positioned at the next available key of `2`, which in this case is `3`.

The iterator should copy `key` and `value` from the block and store them inside the iterator, so that users can access the key and the value without any extra copy with `fn key(&self) -> &[u8]`, which directly returns the reference of the locally-stored key and value.

When `next` is called, the iterator will move to the next position. If we reach the end of the block, we can set `key` to empty and return `false` from `is_valid`, so that the caller can switch to another block if possible.

After implementing this part, you should be able to pass all tests in `block/tests.rs`.

Extra Tasks

Here is a list of extra tasks you can do to make the block encoding more robust and efficient.

Note: Some test cases might not pass after implementing this part. You might need to write your own test cases.

- Implement block checksum. Verify checksum when decoding the block.
- Compress / Decompress block. Compress on `build` and decompress on decoding.

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.